

Final Project

Rotational Gesture Controlled Synthesizer, “My Melody MotionBox”

Cyril El Feghali
ID: 261010517

Lynn Haddad
ID: 261005938

Sofia Velasquez Sierra
ID: 260967314

Walid Aissa
ID: 261039319

Abstract— The "My Melody MotionBox" is a compact and interactive handheld device designed to inspire children to explore STEM (Science, Technology, Engineering, and Mathematics) concepts through motion and sound. By converting simple gestures like tilts and shakes into pitch and tone variations, the synthesizer fosters creativity and curiosity. The device utilizes an STM32 microcontroller, integrating features such as CMSIS for audio synthesis, I2C sensors for motion detection, DAC speakers for sound output, and UART for visual feedback. Real-time audio-visual interaction encourages hands-on learning of sound waves, spatial awareness, and problem-solving skills. Future improvements aim to enhance interactivity with additional sensors and expanded educational content, further solidifying its role as a fun and engaging STEM learning tool.

Keywords— *My Melody MotionBox, STEM education, STM32 microcontroller, audio synthesis, motion detection, real-time feedback, CMSIS, I2C sensor, UART, DAC speaker, OS threading, hands-on learning.*

Introduction

In today's rapidly advancing world, engaging young minds in STEM (Science, Technology, Engineering, and Mathematics) fields is more important than ever. With increasing reliance on technology and engineering solutions, cultivating interest in these subjects from an early age can pave the way for the next generation of innovators. However, traditional teaching methods often fail to capture the imagination of children, leaving a gap in their engagement and understanding. To address this challenge, our team of passionate engineering students sought to create an interactive and educational tool that would make STEM concepts accessible, fun, and hands-on.

This vision led to the creation of the "My Melody MotionBox," a compact handheld device that bridges the gap between learning and play. By allowing children to transform simple movements into sound—adjusting pitch and tone through intuitive gestures like tilts and shakes—our device offers an engaging way to explore sound waves, motion, and spatial awareness. Equipped with real-time audio and visual feedback, the MotionBox encourages creativity, problem-solving, and hands-on exploration, turning abstract STEM

principles into tangible, memorable experiences. Designed with simplicity and accessibility in mind, the device represents an innovative approach to learning, making STEM education both exciting and attainable for young minds.

Part 1: Our Story

We are a group of passionate engineering students who share a common goal: to inspire curiosity and excitement for STEM among children. Recognizing the importance of making STEM concepts more accessible and engaging, we decided to channel our technical knowledge and creativity into a project designed specifically for young learners. With a focus on interactive and hands-on learning, we wanted to create a tool that not only teaches basic principles but also sparks a sense of wonder and discovery. This vision led us to develop a device that bridges the gap between play and education, showing children that science and technology can be both fun and empowering.

Part 2: Our project

The "My Melody MotionBox", or RGCS, is a compact, handheld educational tool designed to make STEM learning exciting and accessible. By converting rotation and motion into sound, it allows users to control pitch and tone through intuitive gestures like tilts and shakes. With real-time audio-visual feedback via speakers and UART, it offers an engaging way to explore sound waves, motion, and spatial awareness. This interactive device not only fosters creativity and imagination but also encourages curiosity and hands-on problem-solving, making STEM concepts tangible and fun for the next generation of innovators.

Why kids will love it

The “My Melody MotionBox” transforms learning into play by combining the magic of sound and movement, all wrapped up in a fun and engaging design. Shaped like a cool gadget or a whimsical instrument, it's easy for kids to pick up and start exploring. Imagine a toy that lets you create music just by tilting, shaking, or spinning it, making every movement come

alive with sound! The device comes with simple, colorful instructions using pictures and friendly language, helping kids quickly understand how it works and why it's exciting. Kids will love experimenting with different gestures to create unique sounds, while gaining a hands-on understanding of how sound waves and motion work together. This concept isn't just fun, it's practical! The device comes with simple, colorful explanations and real-life simple examples that use pictures and friendly language to guide kids through how it works, turning STEM concepts into exciting discoveries. This innovative device bridges the gap between playtime and learning, making STEM concepts fun, accessible, and unforgettable for curious young minds. Perfect for kids who want to discover, create, and imagine!

Part 3: Project Procedure

To bring our idea to life, we developed "My Melody MotionBox" using the STM32 microcontroller as the core of our system. The STM32's powerful architecture and compatibility with various peripherals enabled us to implement features like audio generation, sensor interfacing, and multitasking efficiently.

In our project, we made sure to use the four following features from the list that was given to us and set them up correctly in our IOC:

- CMSIS, DAC Speaker
- I2C Sensor
- OS for threading
- UART

We started by integrating an I2C sensor to capture data from the LSM6DSL gyroscope, enabling precise detection of movement direction and speed. Using the BSP_GYRO_Init() function, the gyroscope was initialized to output three-dimensional angular velocity data, providing real-time feedback on user motion. By continuously sampling the X and Y-axis values, our system dynamically interpreted user gestures, such as tilts and shakes, with high accuracy. This seamless integration of motion data allowed us to translate physical movements into inputs for the synthesizer.

Next, we used CMSIS to generate sine waves for audio output through the DAC speaker. We coded a generate_sine_wave function (Figure 1) to calculate discrete samples of a sine wave using the sine function provided by the library. These samples were scaled to fit within the DAC's output range and stored in dynamically allocated memory to allow for variable sample counts. The waveform's frequency was modulated based on the angular speed measured by the gyroscope on the X and Y axes, with faster movements generating higher frequencies. This modulation was achieved by dynamically adjusting the number of samples per waveform cycle.

The sine wave samples were transmitted to the DAC using DMA which made sure that the playback was non-blocking and efficient. This allowed the system to continue processing user input and generating new sounds in real-time while the DAC played the current waveform.

We configured two channels to produce distinct sine waves: one for movement in the X direction and another for movement in the Y direction. Each channel was routed to a separate speaker, providing auditory feedback that reflects the speed of movement, with faster movements producing higher-pitched sounds and slower movements resulting in lower-pitched tones.

```
void generate_sine_wave(uint32_t* sine_wave_final, int sample_count) {
    // Generate sine values in the range 0 to DAC_SCALE_FACTOR
    for (uint32_t i = 0; i < sample_count; i++) {
        float32_t angle = (2 * PI * i) / sample_count; // Map angle in radians
        float32_t sine_value = arm_sin_f32(angle); // Get sine value

        // Scale and offset to positive range
        sine_wave_final[i] = (uint32_t)((sine_value + 1) * DAC_SCALE_FACTOR / 2);
    }
}
```

Figure 1: "generate_sine_wave" function code

In addition, we utilized UART to offer visual feedback, helping kids better understand the X and Y directions while interacting with the toy. Messages such as "You are moving in the X direction!" were transmitted to help users understand their movements. To tie everything together, we implemented the system using OS threads. A total of four threads were employed: one for reading the sensor values (Figure 2), a second for generating the sine wave for the X direction (Figure 3), a third for the Y direction sine wave (Figure 4), and a fourth for displaying the visual feedback of the direction (Figure 5). This is not trivial threading, as the threads are interconnected and rely on shared states for coordination. The threads communicated through shared variables, with proper synchronization mechanisms to avoid conflicts.

ReadSensorTask: Continuously reads gyroscopic data and updates shared state variables.

GenerateXTask: Processes gyroscopic data for the X-axis, generates a sine wave, and transmits it to the DAC.

GenerateYWaveTask: Processes gyroscopic data for the Y-axis, generates a sine wave, and transmits it to the DAC.

PrintTask: Handles UART output for visual feedback, providing informative messages about movement direction.

```
void ReadSensorTask(void const * argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        osDelay(1);

        // Read Gyroscope (LSM6DSL)
        BSP_GYRO_GetXYZ(gyro);
    }
    /* USER CODE END 5 */
}
```

Figure 2: First OS thread "ReadSensorTask" code

```

void GenerateXTask(void const * argument)
{
    /* USER CODE BEGIN GenerateXTask */
    /* Infinite loop */
    for(;;)
    {
        osDelay(1);
        if (gyro[0] > 10000 || gyro[0] < -10000){
            sine_modulation = 1 + ((abs(gyro[0])-10000)/15000);
            samples = 200 / sine_modulation;

            // Allocate memory for sine wave1
            uint32_t* sine_wavel = (uint32_t*)malloc(samples * sizeof(uint32_t));

            //uint32_t* sine_wavel[samples];

            if (sine_wavel == NULL) {
                // Handle memory allocation failure
                sprintf(buf, "Memory allocation failed\n\r");
                HAL_UART_Transmit(&huart1, (uint8_t*)buf, strlen(buf), 100);
                continue;
            }

            generate_sine_wave(sine_wavel, samples);
            Start_DAC_DMA_Y(samples, sine_wavel);
            osDelay(15);
            HAL_DAC_Stop_DMA(&hdac1, DAC_CHANNEL_2);

            // Free dynamically allocated memory
            free(sine_wavel);
        }
    }
}
/* USER CODE END GenerateXTask */

```

Figure 3: Second OS thread "GenerateXTask" code

```

void GenerateYWaveTask(void const * argument)
{
    /* USER CODE BEGIN GenerateYWaveTask */
    /* Infinite loop */
    for(;;)
    {
        osDelay(1);

        if (gyro[1] > 10000 || gyro[1] < -10000){
            sine_modulation_2 = 1 + ((abs(gyro[1])-10000)/15000);
            samples_2 = 1000 / sine_modulation_2;

            // Allocate memory for sine wave1
            uint32_t* sine_wave2 = (uint32_t*)malloc(samples_2 * sizeof(uint32_t));

            if (sine_wave2 == NULL) {
                // Handle memory allocation failure
                sprintf(buf, "Memory allocation failed\n\r");
                HAL_UART_Transmit(&huart1, (uint8_t*)buf, strlen(buf), 100);
                continue;
            }

            generate_sine_wave(sine_wave2, samples_2);
            Start_DAC_DMA_X(samples_2, sine_wave2);
            osDelay(15);
            HAL_DAC_Stop_DMA(&hdac1, DAC_CHANNEL_1);

            // Free dynamically allocated memory
            free(sine_wave2);
        }
    }
}
/* USER CODE END GenerateYWaveTask */

```

Figure 4: Third OS thread "GenerateYWaveTask" code

```

void PrintTask(void const * argument)
{
    /* USER CODE BEGIN PrintTask */
    /* Infinite loop */
    for(;;)
    {
        osDelay(100);

        if ((gyro[0] > 10000 || gyro[0] < -10000) && (gyro[1] > 10000 || gyro[1] < -10000)){
            sprintf(buf, "Great job kid! You are moving in the X AND Y directions!\n\r");
            HAL_UART_Transmit(&huart1, (uint8_t*)buf, strlen(buf), 100);
        }

        else if (gyro[0] > 10000 || gyro[0] < -10000){
            sprintf(buf, "Great job kid! You are moving in the X direction!\n\r");
            HAL_UART_Transmit(&huart1, (uint8_t*)buf, strlen(buf), 100);
        }

        else if (gyro[1] > 10000 || gyro[1] < -10000){
            sprintf(buf, "Great job kid! You are moving in the Y direction!\n\r");
            HAL_UART_Transmit(&huart1, (uint8_t*)buf, strlen(buf), 100);
        }
    }
}
/* USER CODE END PrintTask */

```

Figure 5: Fourth OS thread "PrintTask" code

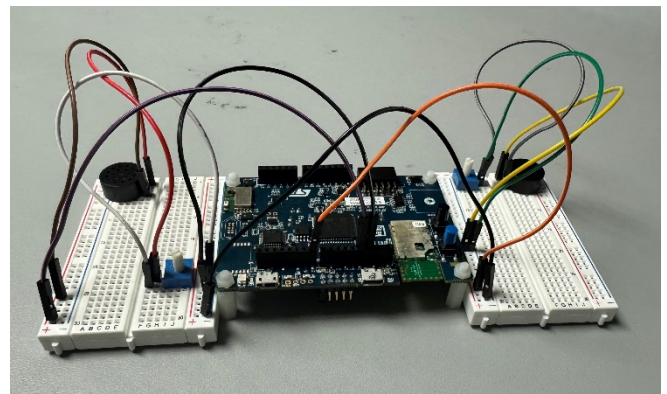


Figure 6: Circuit picture

```

COM3 X
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the X AND Y directions!
Great job kid! You are moving in the Y direction!
Great job kid! You are moving in the X direction!
Great job kid! You are moving in the X direction!
Great job kid! You are moving in the X direction!
Great job kid! You are moving in the X direction!
Great job kid! You are moving in the X direction!
Great job kid! You are moving in the X direction!

```

Figure 7: Visual feedback example

Future Improvements

There are several potential improvements we would like to bring to "My Melody MotionBox" to make it even more interactive and educational. One possibility we are considering is to incorporate additional sensors, such as accelerometers or proximity sensors, to detect a wider range of movements and actions. This would directly enhance the user's experience. Additionally, expanding the educational range of "My Melody MotionBox" could include adding more complex gestures or sound generation techniques, allowing children to explore different aspects of physics, such as frequency modulation or sound synthesis. These improvements would provide a more comprehensive learning experience and encourage deeper engagement with STEM concepts.

Conclusion

The "My Melody MotionBox" successfully integrates motion and sound into an engaging educational tool designed to inspire curiosity about STEM concepts among children. By transforming intuitive gestures such as tilts and shakes into real-time audio-visual feedback, our device bridges the gap between learning and play, making abstract scientific principles tangible and exciting. Through features like the LSM6DSL gyroscope, CMSIS-driven sine wave generation, DAC audio output, UART visual messaging, and OS threading, we showcased how a compact system can deliver a rich, interactive experience.

Our project highlights the power of interdisciplinary engineering, blending hardware design, software development, and educational principles to create a tool that fosters creativity, problem-solving, and hands-on exploration. Beyond its current functionality, the "My Melody MotionBox" has significant potential for growth. Future improvements could include integrating additional sensors to expand the range of motions detected, introducing more complex audio synthesis techniques, and exploring augmented reality to deepen educational engagement. Overall, this project not only demonstrates the technical feasibility of using microcontroller-based systems to enhance STEM education but also features the value of creating tools that inspire the next generation of innovators. By turning movement into music and curiosity into discovery, the "My Melody MotionBox" represents a step forward in making STEM learning accessible, exciting, and impactful for young learners everywhere.